

GRADUATION THESIS

Implementation and Extension of a Visual-Inertial Odometry Pipeline with Hover-Aware FIFO/LIFO Supervisory Estimation

[Candidate Name]

[Department of Robotics / Computer Engineering]

[University Name] · [Year]

Supervisor: [Supervisor Name]

NOTATION AND SYMBOLS

The following table provides a unified reference for all mathematical symbols, frames, and operators used in this thesis. Consistent notation is maintained throughout Chapters 3 and 4.

Table N.1: Mathematical Notation and Symbols

Symbol	Type	Definition
G	Frame	Global (world-fixed) inertial reference frame
I	Frame	IMU body frame, co-moving with the vehicle
C	Frame	Camera optical frame (one instance per camera)
A	Frame	Anchor frame for anchored landmark parameterisations
q_{GtoI}	Quaternion $\in S^3$	JPL unit quaternion encoding rotation from G to I, layout [x,y,z,w]
$R(q)$	Matrix $\in SO(3)$	Rotation matrix corresponding to quaternion q
p_{IinG}	Vector $\in \mathbb{R}^3$	Position of the IMU origin expressed in the global frame
v_{IinG}	Vector $\in \mathbb{R}^3$	Velocity of the IMU frame expressed in the global frame
b_g	Vector $\in \mathbb{R}^3$	Gyroscope bias, modelled as a random walk
b_a	Vector $\in \mathbb{R}^3$	Accelerometer bias, modelled as a random walk
ω_m, a_m	Vectors $\in \mathbb{R}^3$	Raw gyroscope and accelerometer measurements
$\hat{\omega}, \hat{a}$	Vectors $\in \mathbb{R}^3$	Corrected (bias- and intrinsic-compensated) inertial signals
D_w, D_a	Matrices $\in \mathbb{R}^{3 \times 3}$	IMU intrinsic correction matrices (triangular form, model-dependent)
T_g	Matrix $\in \mathbb{R}^{3 \times 3}$	Gravity sensitivity coupling matrix for gyroscope channel
g	Vector $\in \mathbb{R}^3$	Gravity vector expressed in G: $g = [0, 0, g_{mag}]^T$
$\delta\theta$	Vector $\in \mathbb{R}^3$	Orientation error-state (left-multiplicative perturbation)
δx	Vector $\in \mathbb{R}^n$	Full error-state vector
P	Matrix $\in \mathbb{R}^{n \times n}$	Error-state covariance matrix
F	Matrix $\in \mathbb{R}^{15 \times 15}$	IMU error-state transition matrix for one sub-interval

G	Matrix $\in \mathbb{R}^{15 \times 12}$	Noise input mapping matrix
Q_c	Matrix $\in \mathbb{R}^{12 \times 12}$	Continuous-time process noise covariance
Q_d	Matrix $\in \mathbb{R}^{15 \times 15}$	Discrete process noise: $Q_d = G Q_c G^T$
H_x	Matrix	Measurement Jacobian with respect to the state
H_f	Matrix	Measurement Jacobian with respect to a feature position
$\tilde{H}_x$	Matrix	Nullspace-projected measurement Jacobian (feature-free)
$r, \tilde{r}$	Vectors	Innovation vector before and after nullspace projection
S	Matrix	Innovation covariance: $S = \tilde{H}_x P \tilde{H}_x^T + \tilde{R}$
K	Matrix	Kalman gain
σ_{pix}	Scalar	Pixel noise standard deviation
χ^2_{mult}	Scalar	Chi-square gate multiplier (configurable)
$\hat{b}_i$	Unit vector $\in \mathbb{R}^3$	Normalised bearing vector for observation i
d_k	Scalar ≥ 0	Hover decision statistic at frame k (mean or median bearing residual)
ξ_k	Binary $\{0,1\}$	Per-frame binary hover decision
ξ	Binary $\{0,1\}$	Smoothed hover decision (over deque of length <code>hover_smoothing_required</code>)
τ_{hover}	Scalar	Hover detection threshold (<code>hovering_threshold</code>)
q_i	Scalar $\in [0,1]$	LIFO backward-chain quality score for feature i
$\Omega(\omega)$	Matrix $\in \mathbb{R}^{4 \times 4}$	Quaternion kinematic matrix for angular rate ω

CHAPTER 3: SYSTEM ARCHITECTURE

3.1 Overview

The implemented system is a Python-based Visual-Inertial Odometry (VIO) pipeline integrated with the Robot Operating System (ROS). It fuses measurements from an Inertial Measurement Unit (IMU) and one or two cameras to produce real-time six-degree-of-freedom pose estimates, with particular emphasis on robustness during near-stationary hover conditions encountered in Unmanned Aerial Vehicle (UAV) operation.

The estimator follows an error-state Extended Kalman Filter (EKF) architecture derived from the Multi-State Constraint Kalman Filter (MSCKF) paradigm. Two complementary operating modes are supported: a standard FIFO mode in which a sliding window of camera pose clones is maintained and progressively marginalised, and a hover-aware LIFO mode in which clone augmentation is suspended and a deferred covariance strategy is applied. Transitions between modes are governed by a supervisory state machine driven by visual bearing residuals and inertial motion cues.

The runtime is organised around three functional tiers: a high-rate IMU-driven propagation path that publishes low-latency odometry, a camera measurement path that processes frames against available IMU history, and a central manager (VioManager) that orchestrates initialisation, propagation, updates, and mode transitions.

3.2 Runtime Topology and Concurrency Model

Camera processing can execute either synchronously within the IMU callback or asynchronously in a dedicated worker thread, depending on the `use_multi_threading_subs` configuration flag. This distinction is operationally important and is noted here to avoid misrepresentation: the architecture is

best described as a decoupled queue/worker design with optional multithreading, rather than a fixed two-thread pipeline.

Under either mode the structural roles remain consistent. The IMU callback ingests every inertial packet, appends it to the propagator buffer and initialiser feed, and publishes a fast propagated odometry estimate. Camera callbacks are rate-limited and enqueue frames into a timestamp-sorted deque protected by a mutex. The camera worker dequeues a frame only when the IMU buffer contains data sufficiently beyond that frame's timestamp to guarantee accurate interpolation, thereby enforcing causal temporal alignment between the two sensor streams.

Figure 3.1: Queue/worker concurrency model. The IMU callback (left) runs at full sensor rate and triggers the camera worker when idle. The worker (right) drains the sorted camera deque frame-by-frame, processing each against the available IMU history. Dashed arrows indicate optional asynchronous execution.

3.3 Module Decomposition

The pipeline is decomposed into seven major subsystems, each with a clearly defined interface boundary and a single primary responsibility.

3.3.1 ROS Interface Layer (*main.py*)

The ROS wrapper subscribes to IMU and camera topics, converts incoming messages to typed internal data structures (*ImuData*, *CameraData*), enforces per-camera rate limiting, and maintains the sorted camera deque. After each IMU packet, it publishes a fast propagated odometry message and, if the update worker is not already active, spawns or invokes the camera processing routine. Post-update odometry, path history, and feature visualisation images are published through separate channels.

3.3.2 Visual Front-End (TrackKLT)

The front-end tracker implements KLT optical flow over five-level image pyramids with a 15×15 pixel search window. Separate monocular and stereo tracking paths are maintained. Geometric consistency of tentative correspondences is enforced via RANSAC estimation of the fundamental matrix in normalised (undistorted) image coordinates. When the active feature count falls below a threshold, new keypoints are detected using the FAST corner detector on a configurable spatial grid. Optional histogram equalisation and CLAHE preprocessing improve detection reliability in scenes with uneven illumination. Accepted correspondences are committed to the persistent FeatureDatabase.

3.3.3 Feature Database (FeatureDatabase)

The FeatureDatabase maintains per-feature, per-camera observation histories comprising timestamps, distorted pixel coordinates, and undistorted normalised coordinates. Efficient query operators support the back-end scheduling logic: queries for lost tracks (features not observed after a target timestamp), tracks tied to marginalisation events, and tracks spanning required temporal windows for MSCKF constraints. A thread-safe merge utility supports additional measurement insertion from secondary sources.

3.3.4 Inertial Initialiser (InertialInitializer, StaticInitializer)

The initialiser monitors both IMU statistics and visual feature disparity before committing an initial state estimate. Static initialisation gates on accelerometer variance, jerk-split statistics across two temporal half-windows, and bounded visual disparity. Once the gating conditions are satisfied, gravity-aligned orientation, gyroscope bias, and accelerometer bias are estimated and an initial covariance is constructed.

3.3.5 IMU Propagator (Propagator)

The Propagator buffers raw IMU packets and, on request, selects and interpolates the sub-sequence spanning a given time interval. Corrected inertial signals are computed, and the state mean is integrated forward using one of three selectable methods. The error-state transition matrix and process noise are computed for each sub-interval and composed across the full camera interval before a single EKF propagation is applied. Two distinct propagation entry points are provided: `propagate_and_clone` for FIFO operation, which augments the state with a new camera pose clone after propagation, and `propagate_only` for LIFO operation, which updates the IMU state and covariance without modifying the clone window.

3.3.6 Updater Suite

Three independent EKF measurement updaters are provided. `UpdaterMSCKF` applies nullspace-projected constraints from marginalised feature tracks without maintaining feature positions in the filter state. `UpdaterSLAM` supports delayed initialisation and iterative update of persistent landmark variables in the state vector. `UpdaterZeroVelocity` constructs residuals from the stationary-motion assumption and applies a zero-velocity constraint under configurable chi-square and disparity gating.

3.3.7 VIO Orchestrator (VioManager)

`VioManager` is the central coordinator. It directs the tracker, propagator, initialiser, and all three updaters, and it owns the full FIFO/LIFO supervisory state machine. In LIFO mode, it executes the backward-chain feature association pipeline and maintains the deferred measurement buffer for batch covariance reconciliation on FIFO re-entry.

3.4 Coordinate Frames and Conventions

Four reference frames are used consistently throughout the pipeline:

- **G (Global):** A fixed world frame. Gravity is expressed in this frame as $g = [0, 0, g_mag]^T$, pointing in the positive-z direction by convention in this implementation. This vector appears as a subtraction term in the velocity and position dynamics (Section 4.2.2), consistent with code usage.
- **I (IMU Body):** The IMU sensor frame, co-moving with the vehicle platform.
- **C (Camera):** The optical frame of each camera. Camera extrinsics R_ItoC and p_CinI map between the IMU and camera frames and are maintained in the calibration sub-state.
- **A (Anchor):** A selected historical clone pose used as the local reference frame for anchored landmark parameterisations.

All quaternions follow the JPL convention with component layout $[x, y, z, w]$. A quaternion q_GtoI encodes the rotation that maps vectors expressed in the global frame G into the IMU frame I . Orientation error-states use the left-multiplicative small-angle perturbation: for a small rotation error $\delta\theta \in \mathbb{R}^3$, the perturbed quaternion is $\delta q \otimes q_nominal$, where $\delta q \approx [\frac{1}{2}\delta\theta^T, 1]^T$ (unnormalised). This convention yields a 3-DOF orientation error-state, keeping the covariance matrix full-rank.

3.5 End-to-End Data Flow

The processing chain per IMU packet proceeds as follows: (1) the packet is appended to the propagator buffer and, if initialisation has not yet succeeded, also to the initialiser feed; (2) fast-state propagation produces an immediate pose estimate for publication; and (3) the camera worker is triggered if currently idle.

Per camera frame the chain is: (1) the tracker processes the new image, updates correspondences, and commits observations to the FeatureDatabase; (2) the estimator propagates to the camera frame timestamp using either `propagate_and_clone` (FIFO) or `propagate_only` (LIFO); (3) the hover detector evaluates bearing residuals and updates the temporal smoothing deque; (4) the mode transition logic executes; and (5) the appropriate update path runs — either the full MSCKF/SLAM/marginalisation pipeline (FIFO) or the backward-chain state-only update with deferred covariance buffering (LIFO).

3.6 FIFO/LIFO Supervisory State Machine — Architecture View

The two operating modes have fundamentally different clone-window semantics. In FIFO mode, a new camera pose clone is added to the state at every camera frame and the oldest clone is marginalised when the window reaches its maximum size, maintaining a rolling history of at most N_{max} clones. In LIFO mode, clone augmentation is suspended: the clone window is frozen at its current contents and no marginalisation is performed. IMU propagation continues, keeping the IMU sub-state current, but the visual measurement model operates against the static frozen clone context.

Mode transitions are triggered by the smoothed hover decision ξ in combination with velocity and baseline safety guards. Hard exits from LIFO are available via a velocity threshold and a maximum hover duration timer. The detailed transition logic and guard conditions are formalised in Section 4.8.

Figure 3.2: FIFO/LIFO supervisory state machine. The default state is FIFO. Entry into LIFO requires a positive smoothed hover decision and passage of both the velocity and baseline entry guards. Exit from LIFO occurs under any of three conditions: negative hover decision, velocity hard-exit, or maximum duration timeout.

3.7 Outputs and Interfaces

The node exposes two distinct output channels. Fast IMU-rate odometry is derived from the propagated state and published at the full IMU rate, providing low-latency pose estimates between camera updates. Post-update odometry is published after each full EKF update cycle and reflects the corrected filter estimate. An additional path topic accumulates pose history for trajectory visualisation, and a feature image topic overlays tracked keypoints on the camera feed for diagnostic purposes.

3.8 Configuration Surface

All runtime behaviour is parameterised through three YAML files: `estimator_config.yaml` for algorithmic settings, `kalibr_imu_chain.yaml` for IMU noise, intrinsics, and calibration, and `kalibr_imucam_chain.yaml` for camera intrinsics, distortion models, IMU-camera extrinsics, and time-offset calibration. The configuration loader parses these files, constructs the corresponding options objects, and performs a final synchronisation step that copies loaded calibration values into state initialisation structures. A complete configuration key reference is provided in Appendix B.

CHAPTER 4: METHODOLOGY

This chapter presents the complete mathematical and algorithmic methodology of the implemented VIO pipeline. The exposition proceeds from the filter state and error-state formulation (Section 4.1) through IMU propagation (Section 4.2), the visual front-end and triangulation (Sections 4.3–4.4), the MSCKF update (Section 4.5), SLAM landmark management (Section 4.6), zero-velocity updating (Section 4.7), the hover-aware FIFO/LIFO supervisory estimation framework (Section 4.8), and inertial initialisation (Section 4.9). A set of underlying assumptions is stated explicitly in Section 4.10.

4.1 State Representation and Error-State Formulation

4.1.1 Nominal State Vector

The filter tracks a nominal (mean) state vector x partitioned into four logical blocks:

$$x = [x_{IMU} \mid x_{calib} \mid x_{clone,1} \cdots x_{clone,N} \mid x_{L,1} \cdots x_{L,M}]^T \quad (4.1)$$

The mandatory IMU block carries fifteen scalar components:

$$x_{IMU} = [q_{GtoI}, p_{InG}, v_{InG}, b_g, b_a] \quad (4.2)$$

where $q_{GtoI} \in S^3$ is the JPL orientation quaternion (4 parameters, 3 error-state degrees of freedom), $p_{InG} \in \mathbb{R}^3$ is the IMU position in the global frame, $v_{InG} \in \mathbb{R}^3$ is the velocity expressed in the global frame, $b_g \in \mathbb{R}^3$ is the gyroscope bias, and $b_a \in \mathbb{R}^3$ is the accelerometer bias.

When online calibration is enabled via configuration flags, additional sub-states are appended: the camera time-offset $\delta t_{CAMtoIMU}$ (1 DOF), per-camera extrinsic pose T_{ItoC} (6 DOF each), per-camera intrinsics including distortion parameters (8 DOF each), IMU intrinsic correction matrices D_w and D_a (6 DOF each), and gravity sensitivity matrix T_g (9 DOF). The exact set of active calibration blocks is determined at startup and remains fixed throughout the session.

Clone states $x_{\text{clone},i}$ are copies of the IMU pose at camera frame timestamps, added to the state by clone augmentation. SLAM landmark states $x_{L,j}$ hold 3D landmark positions in one of six supported parameterisations described in Section 4.6.2.

4.1.2 Error-State and Covariance

The filter propagates the covariance matrix P of the error-state δx , defined as the difference between the true state and the nominal estimate. For all Euclidean quantities this perturbation is additive; for the orientation quaternion, the left-multiplicative convention is used:

$$\delta q = [\frac{1}{2} \delta\theta_x, \frac{1}{2} \delta\theta_y, \frac{1}{2} \delta\theta_z, 1]^T \text{ (unnormalised)}, \quad q_{\text{true}} = \delta q \otimes q_{\text{nominal}} \quad (4.3)$$

This parameterisation yields a 3-DOF orientation error-state $\delta\theta \in \mathbb{R}^3$ rather than a 4-component quaternion perturbation, keeping P full-rank. The total covariance dimension is:

$$n = n_{\text{IMU}} + n_{\text{calib}} + 6 \cdot N_{\text{clone}} + n_{\text{SLAM}} \quad (4.4)$$

where $n_{\text{IMU}} = 15$, n_{calib} is the sum of all active calibration DOF, N_{clone} is the current clone count, and n_{SLAM} is the total landmark DOF in the state. The covariance $P \in \mathbb{R}^{n \times n}$ is maintained in block form, with explicit cross-covariance sub-matrices between the IMU, clone, and SLAM blocks.

4.1.3 IMU Intrinsic Models

Two IMU intrinsic parameterisations are supported. The KALIBR model stores D_a (and analogously D_w) as a lower-triangular 3×3 matrix, while the RPNG model uses an upper-triangular form. Both are stored as 6-vectors in the state and reconstructed into their respective triangular matrix forms during Jacobian computation. The gravity sensitivity matrix $T_g \in \mathbb{R}^{3 \times 3}$, which models the influence of linear acceleration on the apparent angular rate measurement, is stored as a 9-vector and assembled column-wise when needed.

4.2 IMU Measurement Model and Propagation

4.2.1 Corrected IMU Signals

Raw measurements from the gyroscope ($\omega_m \in \mathbb{R}^3$) and accelerometer ($a_m \in \mathbb{R}^3$) are compensated for bias, scale, misalignment, and gravity sensitivity before integration. The corrected accelerometer signal is:

$$\hat{a} = R_{ACCtoIMU} \cdot D_a \cdot (a_m - b_a) \quad (4.5)$$

The corrected gyroscope signal includes coupling from the gravity sensitivity term:

$$\hat{\omega} = R_{GYROtoIMU} \cdot D_w \cdot (\omega_m - b_g - T_g \cdot \hat{a}) \quad (4.6)$$

The alignment rotations $R_{ACCtoIMU}$ and $R_{GYROtoIMU}$ account for physical misalignment between the sensing axes and the nominal IMU body frame. The gravity sensitivity term $T_g \cdot \hat{a}$ in the gyroscope channel is especially significant for low-cost MEMS sensors, where flexure and vibration-induced coupling can cause substantial apparent angular rate errors.

4.2.2 Continuous-Time Dynamics

The nominal state evolves according to the kinematic equations for a rigid body on the rotation group $SO(3) \times \mathbb{R}^9$:

$$\dot{q} = \frac{1}{2} \Omega(\hat{\omega}) q \quad (4.7)$$

$$\dot{p} = v \quad (4.8)$$

$$\dot{v} = R(q)^T \hat{a} - g \quad (4.9)$$

$$\dot{b}_g = \eta_{wg}, \quad \dot{b}_a = \eta_{wa} \quad (4.10)$$

where $\Omega(\omega) \in \mathbb{R}^{4 \times 4}$ is the skew-symmetric matrix for quaternion kinematics, $R(q) \in SO(3)$ is the rotation matrix corresponding to q_{GtoI} (mapping vectors from G to I), and $g = [0, 0, g_{mag}]^T$ is the gravity vector expressed in the global frame. Note carefully that the term $R(q)^T \hat{a}$ applies the body-to-global rotation to the corrected acceleration, and the subsequent subtraction of g converts specific

force to translational acceleration in the global frame. The quantities η_{wg} and η_{wa} are zero-mean white Gaussian noise processes that drive the bias random walks with spectral densities σ_{wb} and σ_{ab} respectively.

4.2.3 Integration Methods

Three numerical integration schemes are available, selectable via the integration configuration key:

- **Discrete (First-Order Euler):** The quaternion is updated via the large-angle rotation matrix $\Omega(\hat{\omega}, \Delta t)$, and velocity and position are updated with first-order Euler steps. This method is efficient and accurate when the IMU rate is high (≥ 200 Hz) and sub-interval durations are small.
- **Runge-Kutta 4 (RK4):** A four-stage Runge-Kutta scheme is applied to the full kinematic state. Intermediate IMU values are obtained by linear interpolation between the interval endpoints. RK4 substantially reduces integration error for larger time steps, at approximately four times the computational cost of the discrete method.
- **Analytical:** Closed-form integration under the piecewise-constant angular velocity assumption. Precomputed coefficient matrices Ξ_1 through Ξ_4 — derived from the matrix exponential series truncated at closed-form terms — are used for both the mean propagation and the linearised transition Jacobian. This method provides near-exact integration of the rotation kinematics and is used when maximum accuracy is required.

4.2.4 Error-State Transition and Covariance Propagation

For each IMU sub-interval $[t_k, t_{k+1}]$ with duration Δt , the linearised error-state transition matrix $F \in \mathbb{R}^{15 \times 15}$ and noise input mapping matrix $G \in \mathbb{R}^{15 \times 12}$ are computed from the corrected measurements and current nominal state. The discrete process noise covariance is:

$$Q_d = G Q_c G^T \quad (4.11)$$

where $Q_c = \text{diag}(\sigma_{\omega^2} I_3, \sigma_a^2 I_3, \sigma_{wb^2} I_3, \sigma_{ab^2} I_3)$ is the continuous-time noise covariance constructed from the configured noise densities. Interval-level transitions are composed across the full camera interval before a single EKF propagation call is issued:

$$\Phi_{total} = F_n \cdot F_{n-1} \cdot \dots \cdot F_1 \quad (4.12)$$

$$Q_{total} = F_n (\dots (F_2 Q_1 F_2^T + Q_2) \dots) F_n^T + Q_n \quad (4.13)$$

The EKF propagation (StateHelper.EKFPropagation) updates the full covariance block, including cross-covariance sub-matrices between the IMU block and all active clone and SLAM blocks, via the structured product:

$$P_{new} = \Phi P_{old} \Phi^T + Q_{total} \quad (4.14)$$

4.2.5 Clone Augmentation vs. Propagation-Only

After IMU propagation, the pipeline diverges based on the current operating mode. In FIFO mode, `propagate_and_clone` appends a new pose clone to the state by stochastic augmentation: the clone copies the current nominal orientation and position of the IMU, and the covariance is expanded to include full cross-correlations between the new clone block and all existing state variables. When camera time-offset calibration is active, the augmentation additionally injects the covariance coupling between the clone and the time-offset state, preserving the correlation structure that is needed for accurate Jacobian computation in the MSCKF update.

In LIFO mode, `propagate_only` is called instead. The IMU sub-state and all existing covariance blocks are updated by the EKF propagation of Equation (4.14), but no new clone variable is added and the clone window contents remain unchanged. This is the mechanistic foundation of the LIFO hover strategy described in Section 4.8.

4.3 Visual Front-End

4.3.1 KLT Optical Flow Tracking

Feature tracking is performed using the Lucas-Kanade (KLT) pyramidal optical flow algorithm. A five-level image pyramid is constructed for each incoming frame and correspondences are searched within a 15×15 pixel window at each pyramid level. The multi-scale pyramid structure provides robustness across the full range of apparent feature velocities encountered in UAV operation: small

per-pixel displacements during hover and larger displacements during aggressive manoeuvres. Optionally, histogram pre-processing — either global equalisation or Contrast Limited Adaptive Histogram Equalisation (CLAHE) — is applied before pyramid construction to improve detection in scenes with strong illumination gradients or low contrast.

When the number of successfully tracked features falls below a minimum threshold, new features are detected using the FAST corner detector operating on a uniform spatial grid. The grid resolution ($\text{grid_x} \times \text{grid_y}$ cells) and minimum inter-feature distance (min_px_dist pixels) ensure spatially distributed coverage of the image, preventing feature clustering that would reduce the geometric diversity of MSCKF constraints.

4.3.2 Outlier Rejection

After KLT propagation yields a set of tentative correspondences, geometric consistency is enforced via RANSAC estimation of the fundamental matrix. Correspondences are converted to normalised (undistorted) image coordinates before RANSAC to remove the confounding effect of lens distortion on the fundamental matrix geometry. Correspondences whose reprojection error under the estimated fundamental matrix exceeds a configured threshold are marked as outliers and excluded from the FeatureDatabase update. This two-stage pipeline — optical flow tracking followed by geometric verification — provides a robust inlier set suitable for EKF measurement construction.

4.3.3 Feature Database Semantics

Each feature admitted by the outlier rejection stage is inserted into the FeatureDatabase under a persistent integer identity. The database maintains separate per-camera observation arrays of timestamps, distorted pixel coordinates uvs , and undistorted normalised coordinates uvs_norm . The back-end scheduling logic queries the database using three primary operators: `features_not_containing_newer(t)` returns tracks that have not been observed after timestamp t and

are therefore candidates for marginalisation or MSCKF update; `features_containing_older(t)` returns tracks with observations before t , used for SLAM delayed initialisation; and `features_containing(t)` returns tracks spanning a given timestamp, used for clone-window-constrained MSCKF constraints.

4.4 Feature Triangulation and Refinement

4.4.1 Anchor Selection

Before a feature can participate in an EKF update, its 3D position must be estimated. The anchor camera is selected as the camera that has the greatest number of valid observations of the feature; in the case of a tie, the camera with the lower index is preferred. The anchor timestamp is set to the most recent observation in the anchor camera's history. All 3D positions used in anchored parameterisations are expressed relative to this anchor frame.

4.4.2 Linear Triangulation

A linear estimate of the feature position p_{F^A} in the anchor frame is obtained from the bearing orthogonality constraints across all K observations. For each observation i with bearing unit vector $\hat{b}_i$ expressed in the frame of clone pose C_i , the orthogonal projector $B_i^\perp = I - \hat{b}_i \hat{b}_i^T$ enforces that the predicted bearing and the feature position are consistent:

$$A = \sum_i (B_i^\perp)^T B_i^\perp, \quad b = \sum_i (B_i^\perp)^T B_i^\perp p_{\{C_i\}^A} \quad (4.15)$$

$$p_{F^A} = A^{-1} b \quad (4.16)$$

The solution is accepted only if the condition number of A is below a configured threshold and the estimated depth falls within a physically plausible range $[d_{\min}, d_{\max}]$. These checks guard against near-degenerate configurations such as nearly parallel bearing rays, which arise under short-baseline or nearly-coplanar feature layouts.

4.4.3 One-Dimensional Triangulation Variant

For features observed under weak baseline geometry, where the bearing rays are nearly parallel and the full linear system is poorly conditioned, a depth-only triangulation is performed along the anchor bearing $\hat{b}_A$. This one-dimensional solve collapses the three-unknown problem to a single scalar depth λ :

$$\lambda = [\sum_i (B_i^\perp \hat{b}_A)^T (B_i^\perp p_{\{C_i\}^A})] / [\sum_i \|B_i^\perp \hat{b}_A\|^2] \quad (4.17)$$

$$p_{F^A} = \lambda \hat{b}_A \quad (4.18)$$

This variant is numerically more stable than the full triangulation when translational parallax is limited, at the cost of constraining the solution to lie along the anchor bearing direction.

4.4.4 Levenberg-Marquardt Nonlinear Refinement

The linear triangulation provides an initial estimate that is subsequently refined using a Levenberg-Marquardt (LM) iteration over the inverse-depth parameterisation (α, β, ρ) , where the feature position in the anchor frame is $p_{F^A} = (\alpha/\rho, \beta/\rho, 1/\rho)^T$. The LM iteration minimises the sum of squared reprojection errors across all observations. At each iteration, the Hessian H and gradient g are accumulated from per-observation Jacobians, and the parameter update solves:

$$(H + \lambda_{LM} \text{diag}(H)) \Delta\theta = -g \quad (4.19)$$

The LM damping factor λ_{LM} is increased if the step increases the cost and decreased otherwise, providing an adaptive blend between gradient descent and Gauss-Newton steps. Convergence is declared when the norm of the parameter increment $\|\Delta\theta\|$ falls below a configured tolerance or when the maximum iteration count is reached. Final validity checks on depth range and baseline magnitude filter out degenerate refinement results before the feature is passed to the updater.

4.5 MSCKF Measurement Update

4.5.1 Principle and Motivation

The Multi-State Constraint Kalman Filter (MSCKF) provides visual constraints on the clone poses without augmenting the state with feature positions. A feature track spanning K camera clones contributes $2K$ pixel observations linked to K clone pose variables already present in the state. The feature's 3D position is eliminated from the linearised system by a nullspace projection, yielding a set of pure state constraints. This approach avoids the quadratic covariance growth that would occur if every tracked feature were added as a landmark variable, making the MSCKF computationally tractable for real-time operation with hundreds of features.

4.5.2 Measurement Jacobian Construction

For a feature f with estimated global position p_{F^G} observed at clone i with camera pose $(R_{\{GtoI_i\}}, p_{\{I_i\}^G})$ and camera-IMU extrinsics (R_{ItoC}, p_{CinI}) , the predicted observation through the camera projection and distortion model $\pi(\cdot)$ is:

$$\hat{z}_i = \pi(R_{ItoC} (R_{\{GtoI_i\}} (p_{F^G} - p_{\{I_i\}^G}) - p_{CinI})) \quad (4.20)$$

Linearising the reprojection residual $r_i = z_i - \hat{z}_i$ around the current nominal state yields the observation Jacobians:

$$r_i \approx H_{\{x,i\}} \delta x + H_{\{f,i\}} \delta f + n_i \quad (4.21)$$

where $H_{\{x,i\}} \in \mathbb{R}^{2 \times n}$ is the Jacobian with respect to the full state, $H_{\{f,i\}} \in \mathbb{R}^{2 \times 3}$ is the Jacobian with respect to the feature position, and $n_i \sim N(0, \sigma_{\text{pix}}^2 I_2)$ is the pixel noise. When camera extrinsic or intrinsic calibration is active, additional columns are included in $H_{\{x,i\}}$ corresponding to the relevant calibration sub-state variables. Stacking over all K observations produces the system-level representation:

$$r \approx H_x \delta x + H_f \delta f + n \quad (4.22)$$

4.5.3 Nullspace Projection

The feature Jacobian $H_f \in \mathbb{R}^{2K \times 3}$ is factored using a thin QR decomposition: $H_f = [Q_1, Q_2] [T; 0]$, where $Q_1 \in \mathbb{R}^{2K \times 3}$ and $Q_2 \in \mathbb{R}^{2K \times (2K-3)}$ form an orthonormal basis with Q_2 spanning the left nullspace of H_f^T . Pre-multiplying Equation (4.22) by Q_2^T eliminates the feature term:

$$\tilde{r} = Q_2^T r, \quad H_x = Q_2^T H_x, \quad \tilde{n} = Q_2^T n \quad (4.23)$$

The projected residual $\tilde{r} \approx \tilde{H}_x \delta x + \tilde{n}$ is a pure state constraint. The projected noise $\tilde{n}$ is Gaussian with covariance $\tilde{R} = \sigma_{\text{pix}}^2 Q_2^T Q_2 = \sigma_{\text{pix}}^2 I$, since Q_2 has orthonormal columns. Multiple features are processed individually and their projected constraints are stacked into a batch update matrix.

4.5.4 Chi-Square Innovation Gating

Each feature's projected system is validated before stacking. The innovation covariance is:

$$S = H_x P_{\text{marg}} H_x^T + R \quad (4.24)$$

where P_{marg} is the marginal covariance over the state variables appearing in $\tilde{H}_x$. The Mahalanobis distance is computed and compared against the chi-square critical value at the 95th percentile with the appropriate degrees of freedom:

$$\tilde{r}^T S^{-1} \tilde{r} \leq \chi^2_{\text{mult}} \cdot \chi^2_{\{0.95, \text{dof}\}} \quad (4.25)$$

Features failing the gate, features with poorly conditioned innovation matrices (condition number exceeding $\text{max_innovation_condition}$), or features producing non-finite Jacobians are excluded. This gating rejects outlier tracks caused by triangulation errors, dynamic objects, or brief occlusions without contaminating the filter estimate.

4.5.5 Measurement Compression and EKF Update

After gating, all accepted projected systems are stacked and a QR compression step reduces the row dimension of the combined system to at most n , where n is the state dimension. This compression

preserves the information content of the constraint while reducing the computational cost of the Kalman gain solve. The standard EKF update is then applied:

$$K = P H_x^T (H_x P H_x^T + R)^{-1} \quad (4.26)$$

$$\delta x = K \tilde{r} \quad (4.27)$$

$$P \leftarrow (I - K H_x) P \quad (4.28)$$

The innovation covariance S is solved via Cholesky decomposition with a small diagonal jitter for numerical stability. The state increment δx is applied on the appropriate manifold for each variable: left-multiplicative quaternion update for orientations, and additive increments for all Euclidean quantities. Covariance symmetry is enforced by post-update averaging.

4.6 SLAM Landmark Management

4.6.1 Delayed Initialisation

Long-lived tracks that accumulate sufficient observations and pass triangulation quality checks are promoted from the MSCKF pool to persistent SLAM landmarks in the state vector. The promotion uses a gated initialisation procedure (`StateHelper.initialize`): the linearised relationship between the new landmark block and the existing state is computed from the triangulation Jacobian, and the state and covariance are expanded accordingly. A minimum track duration (`dt_slam_delay`) prevents premature initialisation of features observed under a single geometric configuration.

4.6.2 Landmark Representations

Six coordinate parameterisations are supported for SLAM landmarks, each offering a different trade-off between DOF count, numerical conditioning, and Jacobian complexity:

- **GLOBAL_3D**: Euclidean coordinates in the global frame (3 DOF). Simple but susceptible to poor conditioning for distant or weakly-parallaxed features.

- GLOBAL_FULL_INVERSE_DEPTH: Full inverse-depth representation in the global frame (6 DOF).
- ANCHORED_3D: Euclidean coordinates relative to an anchor clone pose (3 DOF).
- ANCHORED_FULL_INVERSE_DEPTH: Inverse-depth relative to an anchor (6 DOF), the most expressive form.
- ANCHORED_MSCKF_INVERSE_DEPTH: Compact anchored inverse-depth form (3 DOF) aligned with MSCKF usage.
- ANCHORED_INVERSE_DEPTH_SINGLE: Single-depth parameterisation (1 DOF) with a cached anchor bearing vector. Used for nearly-static or deeply-anchored features.

4.6.3 Anchor Management

When the oldest clone in the sliding window is marginalised during FIFO operation, any SLAM landmark anchored to that clone must be re-anchored. The anchor change computes the relative pose transformation between the old and new anchor frames, converts the landmark coordinates through this transformation, and propagates the covariance change through the corresponding Jacobian. This operation preserves filter consistency under anchor transitions and prevents the introduction of spurious correlations that would arise from an inconsistent re-parameterisation.

4.7 Zero-Velocity Update

The Zero-Velocity Update (ZUPT) provides strong observability constraints when the vehicle is stationary or near-stationary. Under true stationarity, the IMU measurements reflect only bias effects and the projection of gravity, and the velocity should be zero. The ZUPT exploits this to construct EKF measurements that enforce the stationary assumption.

4.7.1 Residual Model

Over a selected IMU window of duration Δt , two residual types are constructed. The gyroscope residual enforces zero angular rate:

$$r_{\omega} = -(\sqrt{\Delta t}/\sigma_{\omega}) \hat{\omega} \quad (4.29)$$

The accelerometer residual enforces consistency between the measured specific force and the gravity vector projected into the IMU frame:

$$r_a = -(\sqrt{\Delta t} / \sigma_a) (\hat{a} - R_{\{GtoI\}} g) \quad (4.30)$$

Both residuals are normalised by their respective measurement noise densities to yield dimensionless innovations with unit covariance. The Jacobians of these residuals couple primarily to the orientation and bias sub-states. It is noted that the velocity sub-state is not always included in the Jacobian structure: the default residual formulation does not include velocity sensitivity, and the velocity block is only engaged when the integrated-acceleration ZUPT variant is enabled in the updater options.

4.7.2 Acceptance Gating

The ZUPT update is applied only when all three of the following conditions are simultaneously satisfied: (1) the chi-square test on the stacked residual passes under the configured multiplier `zupt_chi2_multiplier`; (2) the norm of the current estimated velocity is below `zupt_max_velocity`; and (3) the visual disparity computed from a reference feature pair is below `zupt_max_disparity`. The disparity condition is critical for preventing ZUPT from firing during slow rotational motion — particularly relevant for UAV hover — where the IMU signal might pass the chi-square test while the camera is observing genuine translational or rotational motion. A force mode is available for use at startup, in which the disparity check is mandatory while the chi-square test is relaxed.

4.8 Hover-Aware FIFO/LIFO Supervisory Estimation Framework

This section describes the primary original contribution of this thesis. The motivation, hover detection methodology, mode switching logic, LIFO update semantics, and exit reconciliation procedure are described in order.

4.8.1 Motivation: Observability Degradation Under Hover

In standard FIFO MSCKF operation, translational observability is derived from the geometric parallax accumulated between successive camera clone poses. As the UAV transitions to near-stationary hover, the inter-frame camera baseline collapses toward zero and the parallax of all observed features diminishes. In this regime, the feature Jacobian H_f approaches rank deficiency in the translational subspace, and the nullspace-projected constraint $\tilde{H}_x$ provides negligible information about the translational state components. Repeated application of near-zero-information updates introduces numerical noise into P without genuine estimation benefit, and may lead to gradual covariance degradation.

Rather than continuing to process degenerate MSCKF constraints during hover, the proposed framework detects the onset of low-excitation conditions and transitions to a hover-specialised LIFO mode that suspends clone augmentation, applies a state-only mean update, and defers covariance changes for batch reconciliation on motion resumption. This design preserves the filter's covariance structure through the hover period while continuing to extract whatever translational information the frozen clone window affords.

4.8.2 Hover Detection Statistic

Hover is detected at each camera frame by computing a bearing residual statistic that quantifies apparent feature displacement after compensating for IMU-estimated rotation. Given normalised bearing vectors $b_{\{prev,i\}}$ and $b_{\{curr,i\}}$ observed in two consecutive frames, and the relative rotation matrix $C_{\{\hat{q}\}}$ derived from IMU propagation between the frame timestamps, the per-feature residual is:

$$r_i = \|b_{\{curr,i\}} - C_{\{\hat{q}\}} b_{\{prev,i\}}\|^2 \quad (4.31)$$

After rotation compensation, any residual displacement in $b_{\text{curr},i}$ relative to $C_{\{\hat{q}\}} b_{\{\text{prev},i\}}$ must be attributable to translational camera motion. Under pure hover, this residual should be negligibly small for all features; under translational motion it grows with baseline.

4.8.3 Baseline-Adaptive Inlier Filtering

The set of bearings used to compute the decision statistic is selected adaptively based on the estimated inter-frame camera baseline magnitude b_{cam} , derived from the difference in clone-pose positions propagated through the camera extrinsics. Two policies are applied:

- **Large-baseline regime ($b_{\text{cam}} > \text{hover_baseline_threshold}$):** Epipolar geometry filtering is applied. Correspondences inconsistent with the estimated essential matrix are removed. This prevents genuinely large-motion frames from producing falsely small residuals due to degenerate feature configurations near the epipole.
- **Small-baseline regime ($b_{\text{cam}} \leq \text{hover_baseline_threshold}$):** A direct bearing residual threshold filter is applied, retaining only correspondences whose residual r_i falls below $\text{hover_bearing_inlier_threshold}$.

The decision statistic d_k is then computed over the filtered inlier set. When the inlier count is sufficient, the mean residual is used:

$$d_k = (1/|\mathcal{I}_k|) \sum_{i \in \mathcal{I}_k} r_i \quad (4.32)$$

When the inlier count falls below a minimum threshold hover_min_inliers , a median over the full residual set provides a more robust fallback:

$$d_k = \text{median}\{r_i : i \in \text{all correspondences}\} \quad (4.33)$$

The binary hover decision is then:

$$\xi_k = 1 \text{ if } d_k < \tau_{\text{hover}}, \quad \xi_k = 0 \text{ otherwise} \quad (4.34)$$

4.8.4 Temporal Smoothing and Mode Entry Guards

To suppress mode flapping from transient feature tracking noise, binary decisions are accumulated in a fixed-length deque of size $\text{hover_smoothing_required}$. A FIFO→LIFO transition is considered

only when all entries in the deque are 1, indicating consistent hover detection over multiple consecutive frames. If the deque has not yet filled (e.g., at startup), the last stable decision is retained.

The smoothed hover decision ξ is 1 only when this full consistency condition holds.

Even with $\xi = 1$, the FIFO→LIFO transition is blocked unless two safety guards simultaneously pass:

- **Velocity guard:** $\|v_{inG}\| \leq \text{hover_entry_max_velocity}$. Prevents LIFO entry during slow translational glides that may produce low bearing residuals.
- **Baseline guard:** $b_{cam} \leq \text{hover_entry_max_baseline}$. Ensures the camera is genuinely in a near-static geometric configuration.

The complete FIFO→LIFO entry condition is therefore: $(\xi = 1) \text{ AND } (\|v\| \leq v_{entry_max}) \text{ AND } (b_{cam} \leq b_{entry_max})$.

4.8.5 LIFO→FIFO Exit Conditions

The system returns to FIFO mode under any of the following three conditions, evaluated at each camera frame while in LIFO:

1. The smoothed hover decision ξ drops to 0, indicating that translational motion has resumed and the bearing residuals have grown beyond the threshold over the full smoothing window.
2. The instantaneous velocity magnitude $\|v_{inG}\|$ exceeds $\text{hover_exit_velocity_threshold}$. This hard exit operates independently of the visual detector and provides a fast-response fallback when IMU-detected motion precedes visible feature displacement.
3. The cumulative duration spent in LIFO mode exceeds $\text{hover_max_duration_sec}$. This unconditional timer prevents indefinite hover operation in cases where the visual detector fails to trigger an exit.

4.8.6 LIFO Update Semantics: Backward-Chain Feature Association

Within LIFO mode, the frozen clone window provides a stable reference structure against which current feature observations can be associated. Since no new clone is added to the state, feature association must work backwards against the existing clones — hence the designation backward-chain.

For each candidate feature, a quality score $q_i \in [0, 1]$ is maintained and updated each frame according to the outcome of the association attempt:

$$q_i \leftarrow q_i + \Delta q_{match} \quad (\text{successful geometric match}) \quad (4.35)$$

$$q_i \leftarrow q_i + \Delta q_{ORB} \quad (\text{successful ORB descriptor fallback match}) \quad (4.36)$$

$$q_i \leftarrow q_i - \Delta q_{miss} \quad (\text{failed match or excess misses}) \quad (4.37)$$

Features are pruned from the active chain when $q_i < q_{min_keep}$ or when the miss count exceeds $lifo_max_misses$. Only features with $q_i \geq q_{min_output}$ and $chain_length \geq lifo_backward_min_chain_length$ are passed to the MSCKF updater within LIFO mode.

The assignment of current observations to backward chain entries is solved using the Hungarian algorithm applied to a cost matrix with entries:

$$cost(i, j) = w_r \cdot cost_{reproj}(i, j) + w_d \cdot cost_{desc}(i, j) \quad (4.38)$$

where $cost_{reproj}$ is the normalised reprojection cost and $cost_{desc}$ is the normalised ORB Hamming descriptor distance. Descriptor matching is applied as a conditional fallback rather than universally: it engages only when the geometric reprojection cost alone is insufficient to resolve ambiguous assignments or when `lifo_orb_descriptor_fallback` is explicitly enabled. An optional mutual consistency check rejects assignments where the forward-backward assignment pair is not mutually nearest-neighbour within the assignment solution, suppressing false associations in feature-crowded image regions.

4.8.7 Deferred Covariance Strategy

A defining design decision in LIFO mode is the intentional decoupling of mean and covariance updates. At each LIFO frame, the MSCKF update is applied to the state mean as normal — producing a corrected nominal state that reflects the visual measurements — but the resulting posterior

covariance is not retained. Instead, the pre-update covariance snapshot is restored, and the current frame's projected measurement system ($\tilde{H}_x, \tilde{r}$) is appended to a deferred measurement buffer.

The rationale is as follows. Under near-zero-baseline hover, the MSCKF Jacobians $\tilde{H}_x$ are geometrically degenerate in the translational subspace: the update provides essentially no information about position or depth, but the formal EKF covariance update would still modify P in potentially inconsistent ways due to the degenerate matrix inversion. Accumulating these degenerate updates over many LIFO steps could render P overconfident in translational directions and cause filter inconsistency upon FIFO re-entry. Deferring the covariance update and applying the accumulated measurements as a single batch at FIFO re-entry avoids this accumulation and preserves a conservative, consistent uncertainty estimate through the hover period.

It is acknowledged that strict simultaneous mean-covariance coupling — as required by the standard EKF derivation — is relaxed by this strategy. This constitutes an engineering approximation motivated by the specific geometry of the hover regime, and its effect on filter consistency should be characterised through NEES analysis in the experimental evaluation (Section 6.3).

4.8.8 FIFO Re-Entry and Covariance Reconciliation

Upon a LIFO→FIFO mode transition, the deferred measurement buffer accumulated during the hover period is processed in a single batch covariance reconciliation step. The stacked projected systems from the buffer are applied to the restored covariance using the standard EKF update equations (4.26–4.28). This condenses the geometric information gathered during hover into a single consistent covariance correction before normal FIFO clone augmentation resumes. The state mean is handled through the snapshot mechanism as implemented in the VioManager reconciliation logic, and subsequent FIFO updates proceed from the reconciled filter state.

Table 4.1: Comparison of FIFO and LIFO Operating Mode Characteristics

Property	FIFO Mode	LIFO Mode
Clone augmentation	New clone appended each frame	Clone window frozen
Propagation path	propagate_and_clone	propagate_only
MSCKF update	Full standard update	State-only; covariance deferred
Feature association	Forward KLT tracking	Backward chain, quality-gated
Covariance update	Applied at each step	Snapshot restored; batch on exit
SLAM updates	Active	Suspended
Clone marginalization	Oldest clone dropped at capacity	Not performed
Entry condition	Default / LIFO exit event	$\xi = 1$ AND velocity AND baseline guards
Exit condition	—	$\xi = 0$ OR velocity hard-exit OR timer

4.9 Inertial Initialisation

4.9.1 Initialisation Gating

The filter cannot begin estimation until the initial orientation, velocity, and IMU biases have been recovered. A static initialisation procedure is employed that gates on both inertial statistics and visual motion cues to prevent premature or erroneous initialisation.

The IMU window of duration `init_window_time` is split into two temporal halves. The accelerometer variance is computed in each half, and the variance between-halves is compared against a jerk threshold. Initialisation is attempted only when the intra-window variance is below `init_imu_thresh` in both halves, confirming near-stationary conditions, and the visual disparity computed from the `FeatureDatabase` is below `init_max_disparity`, confirming that the camera is also stationary. If either condition fails, the check is deferred and retried on the next camera frame.

4.9.2 Gravity-Aligned Orientation Recovery

Under static conditions, the mean accelerometer reading $\bar{a}_m$ approximates the gravity vector projected into the IMU frame: $\bar{a}_m \approx R_{\{GtoI\}} g$. A gravity-aligned initial orientation $R_{\{GtoI,0\}}$ is recovered by constructing an orthonormal basis via Gram-Schmidt orthogonalisation that aligns the measured acceleration direction with the known gravity vector expressed in the global frame.

4.9.3 Initial Bias Estimation

With the gravity-aligned orientation available, the initial biases are estimated as:

$$b_{g,0} = \bar{\omega}_m \quad (\text{mean gyroscope reading under stationary assumption}) \quad (4.39)$$

$$b_{a,0} = \bar{a}_m - R_{\{GtoI,0\}} g \quad (\text{specific-force residual after gravity removal}) \quad (4.40)$$

The initial velocity is set to zero. The initial state covariance is constructed from fixed uncertainty values specified in the configuration, reflecting the accuracy of the gravity-alignment procedure and the variance of the window-averaged bias estimates.

4.10 Underlying Assumptions

The methodology described in this chapter rests on the following explicit assumptions. Violations of these assumptions may degrade estimation accuracy and should be considered in evaluating system performance.

- A.1 IMU measurements are corrupted by additive white Gaussian noise and slow-varying random-walk biases. Time-correlated or higher-order noise processes are not modelled.
- A.2 The camera-IMU time offset $\delta t_{\text{CAMtoIMU}}$ is either precisely known from calibration or estimated online as part of the filter state. Unsynchronised clocks with nonlinear drift are not handled.
- A.3 Camera and IMU extrinsic calibration is approximately known from an offline calibration procedure and, if not estimated online, remains fixed throughout the session.
- A.4 The scene is predominantly static. Dynamic objects that violate the static-world assumption are treated as outliers and are expected to be rejected by RANSAC and the chi-square innovation gate.

- A.5 The vehicle experiences sufficient translational excitation during FIFO operation to maintain translational observability. Sustained pure rotation under FIFO mode degrades position estimation quality, independently of the hover framework.
- A.6 During hover, the dominant motion is translational quiescence rather than rotation. The hover detector is designed to respond to translational bearing residuals and may not correctly classify sustained in-place rotation as a hover condition.
- A.7 The deferred covariance strategy in LIFO mode introduces an approximation to the standard EKF. It is assumed that the geometric degeneracy of the hover configuration is sufficiently severe that the information lost by covariance deferral is negligible relative to the consistency benefit of avoiding degenerate covariance updates. This assumption requires empirical validation via NEES analysis.

CHAPTER 5: IMPLEMENTATION (OUTLINE)

Chapter 5 documents the Python-specific engineering decisions that distinguish this implementation from a direct translation of the methodology and from the original OpenVINS C++ codebase. The following subsections are recommended.

5.1 Python Port Rationale and Constraints

This subsection should discuss the choice of Python as the implementation language, the implications of the Global Interpreter Lock (GIL) on callback throughput, and the use of NumPy and SciPy for vectorised linear algebra in place of Eigen-based C++ operations. It should quantify any observed performance difference relative to a C++ baseline and describe the mitigation strategies adopted (e.g., optional asynchronous threading, rate-limited camera processing).

5.2 JPL Quaternion and Lie Algebra Library

Describe the implementation of `quat_ops.py`: the rotation group and Lie algebra utilities `exp_so3`, `log_so3`, `Jr_so3`, and their use in the EKF update and initialisation procedures. Note any numerical stability choices relative to standard reference implementations.

5.3 Dynamic State and Covariance Management

Explain the Python dictionary-based state variable ordering in `State.py` and `StateOptions.py`, contrasted with the compile-time type registration in C++ OpenVINS. Describe how `StateHelper.EKFPropagation` dynamically resolves block indices for variable-length covariance updates.

5.4 Camera Model Implementations

Document the CamRadtan and CamEqui class hierarchy, the projection and unprojection interfaces used in Jacobian chains, and any numerical differences from the reference model implementations.

5.5 FEJ Implementation

Describe how First-Estimates Jacobians are stored in JPLQuat and Landmark objects and applied in the updater Jacobian paths when `state_options.do_fej` is enabled.

5.6 Dynamic Initialiser Status

State clearly that `initialize/dynamic.py` is present in the repository but contains no implementation.

Only static initialisation is available in the current system. This is a known scope boundary.

CHAPTER 6: EXPERIMENTS AND EVALUATION (OUTLINE)

Chapter 6 must provide quantitative evidence of system correctness and the contribution of the hover-aware framework. The recommended structure and content are as follows.

6.1 Dataset Characterisation

Each evaluation sequence should be characterised along three dimensions: the motion regime (proportion of time in hover, slow drift, aggressive manoeuvre), scene texture level (dense, sparse, textured, near-planar), and lighting or blur conditions (consistent, variable, motion-blurred). This characterisation allows the hover framework contribution to be assessed against sequences where hover mode is expected to engage versus sequences dominated by dynamic motion.

6.2 Reproducibility and Experiment Protocol

The experiment section should document the exact configuration file and commit hash used, the dataset split policy (if sequences are divided for tuning vs. evaluation), any random seed control, and the hardware and software environment (processor, memory, ROS version, Python version, OpenCV version). This information is required for reproducibility.

6.3 Evaluation Metrics

Table 6.1: Evaluation Metrics

Metric	Description	Tool / Method
ATE (RMSE)	Absolute Trajectory Error — RMSE after optimal SE(3) alignment	evo_ape
RTE	Relative Trajectory Error — drift per unit length over fixed-length segments	evo_rpe
NEES	Normalised Estimation Error Squared — filter consistency measure. Expected value: degrees of freedom.	Custom script

NIS	Normalised Innovation Squared — per-update consistency check.	Custom script
Hover occupancy (%)	Fraction of camera frames processed in LIFO mode	Mode log
Update acceptance rate	Fraction of MSCKF batch updates accepted by the chi-square gate	Timing log
Per-module latency (ms)	Average and peak computation time per module per frame	Timing log

The NEES computation protocol should state the sampling window length, the confidence interval (typically 95%), and the expected upper/lower bounds for a consistent filter. If NEES systematically exceeds the upper bound, the filter is overconfident; if it falls below the lower bound, the filter is underconfident. Both conditions are informative about the approximations introduced by the deferred covariance strategy.

6.4 Ablation Study

The following four configurations are recommended to isolate the contribution of each system extension:

Table 6.2: Ablation Study Configurations

Configuration	LIFO Enabled	ZUPT Enabled	Purpose
Full system (A)	Yes	Yes	Baseline for all comparisons
No hover mode (B)	No	Yes	Isolates contribution of FIFO/LIFO switching
No ZUPT (C)	Yes	No	Isolates ZUPT contribution from hover handling
MSCKF only (D)	No	No	Nearest equivalent to canonical OpenVINS behaviour

6.5 Sensitivity Analysis

The hover framework introduces several threshold parameters whose tuning affects system behaviour. A planned sensitivity sweep over the following parameters is recommended to characterise robustness: `hovering_threshold` ($\pm 20\%$ from nominal), `hover_smoothing_required` (2, 3, 5 frames), `lifo_quality_min_output` (0.2, 0.4, 0.6), and `zupt_max_disparity`. Each parameter should be swept on a held-out validation sequence and the resulting ATE and hover-occupancy changes reported.

CHAPTER 7: DISCUSSION (OUTLINE)

Chapter 7 should interpret the experimental results in the context of the methodology. Recommended subsections:

7.1 Performance Analysis

Interpret ATE/RTE results per sequence and per ablation configuration. Identify which sequences benefit most from the hover framework and explain the connection to the motion regime characterisation of Section 6.1.

7.2 Filter Consistency

Analyse NEES/NIS trends. If the deferred covariance strategy produces no significant NEES degradation, this supports the assumption stated in Section 4.10 (item 7). If NEES degradation is observed on hover-heavy sequences, quantify its magnitude and discuss its practical significance.

7.3 Failure Mode Analysis

Document observed failure modes: mode flapping at the hover threshold boundary, backward-chain degradation under motion blur, ZUPT false triggers during slow rotation, and propagation-only drift during extended LIFO periods. For each failure mode, describe the conditions under which it occurs and the tuning mitigations available.

7.4 Threats to Validity

Discuss limitations of the evaluation: dataset bias toward specific motion profiles, calibration drift between recording sessions, the absence of a dynamic initialiser, and potential performance differences on hardware platforms with different IMU noise characteristics. Acknowledge the scope boundaries of the static-world assumption.

7.5 Computational Performance

Report per-module timing results from Section 6.2. Discuss the performance gap relative to C++ OpenVINS and identify the dominant cost centres (e.g., covariance block operations, KLT pyramid construction, Hungarian assignment in LIFO mode).

CHAPTER 8: CONCLUSION AND FUTURE WORK

8.1 Summary of Contributions

This thesis has presented the design, implementation, and evaluation of a complete Visual-Inertial Odometry pipeline in Python, targeting UAV platforms with hover-intensive operation profiles. The primary technical contributions are:

- C.1 A full Python reimplementation of an MSCKF-based VIO pipeline, including IMU propagation with three selectable integration methods, SLAM delayed initialisation and anchor management, ZUPT with visual disparity gating, and static inertial initialisation.
- C.2 A hover-aware supervisory FIFO/LIFO estimation framework comprising: a bearing-residual hover detector with baseline-adaptive inlier filtering and temporal smoothing, a guarded mode transition state machine, a backward-chain feature association algorithm with Hungarian matching and quality-score evolution, a deferred covariance strategy for near-degenerate hover geometry, and a batch covariance reconciliation procedure on FIFO re-entry.

8.2 Future Work

Several directions for future development are identified. Implementation of the dynamic initialisation module (currently an empty stub) would enable deployment on platforms that cannot satisfy the static initialisation requirements. An online NEES monitoring mechanism that dynamically adjusts the LIFO quality thresholds based on observed filter consistency would reduce the need for manual threshold tuning. Extension of the hover detector to explicitly handle in-place rotation, which the current bearing residual formulation cannot reliably classify, would improve robustness under complex hover manoeuvres. Finally, a theoretical analysis of the deferred covariance approximation error under parameterisable near-degenerate geometry would strengthen the methodological justification of the LIFO update strategy.

REFERENCES

The following references constitute the minimum required citation set for this thesis. Additional citations should be added as the related-work and background chapters are developed.

- [1] Mourikis, A. I. and Roumeliotis, S. I. (2007). A Multi-State Constraint Kalman Filter for Vision-Aided Inertial Navigation. *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*, pp. 3565–3572.
- [2] Geneva, P., Eckenhoff, K., Lee, W., Yang, Y., and Huang, G. (2020). OpenVINS: A Research Platform for Visual-Inertial Estimation. *Proc. IEEE Int. Conf. Robotics and Automation (ICRA)*.
- [3] Forster, C., Carlone, L., Dellaert, F., and Scaramuzza, D. (2017). On-Manifold Preintegration for Real-Time Visual-Inertial Odometry. *IEEE Trans. Robotics*, 33(1), pp. 1–21.
- [4] Huang, G., Mourikis, A. I., and Roumeliotis, S. I. (2010). Observability-based Rules for Designing Consistent EKF SLAM Estimators. *Int. J. Robotics Research*, 29(5), pp. 502–528.
- [5] Li, M. and Mourikis, A. I. (2013). High-Precision, Consistent EKF-Based Visual-Inertial Odometry. *Int. J. Robotics Research*, 32(6), pp. 690–711.
- [6] Lucas, B. D. and Kanade, T. (1981). An Iterative Image Registration Technique with an Application to Stereo Vision. *Proc. Int. Joint Conf. Artificial Intelligence (IJCAI)*, pp. 674–679.
- [7] Kuhn, H. W. (1955). The Hungarian Method for the Assignment Problem. *Naval Research Logistics Quarterly*, 2(1–2), pp. 83–97.
- [8] Furgale, P., Rehder, J., and Siegwart, R. (2013). Unified Temporal and Spatial Calibration for Multi-Sensor Systems. *Proc. IEEE/RSJ Int. Conf. Intelligent Robots and Systems (IROS)*.
- [9] Hartley, R. and Zisserman, A. (2004). *Multiple View Geometry in Computer Vision*, 2nd ed. Cambridge University Press.
- [10] Chirikjian, G. S. (2012). *Stochastic Models, Information Theory, and Lie Groups*, Vol. 2. Birkhäuser.

APPENDIX A: MATHEMATICAL DERIVATIONS

A.1 JPL Quaternion Left-Multiplicative Update

For a small rotation error $\delta\theta \in \mathbb{R}^3$ in the filter error-state, the perturbed quaternion is constructed as follows. Define the error quaternion:

$$\delta q \approx [\frac{1}{2} \delta\theta^T, 1]^T \quad (\text{unnormalised, valid for } \|\delta\theta\| \ll 1) \quad (\text{A.1})$$

The corrected nominal quaternion is:

$$q^+ = \text{quatnorm}(\delta q \otimes q_{\text{nominal}}) \quad (\text{A.2})$$

Under the JPL convention, the Hamilton product $\delta q \otimes q$ applies the incremental rotation δq to the left (body-frame side) of q . Normalisation ensures the result lies on S^3 . This convention is consistent throughout the filter: all Jacobians in $H_{\{x,i\}}$ and F use the same left-multiplicative orientation error.

A.2 Quaternion Kinematics Matrix

The quaternion kinematic differential equation $\dot{q} = \frac{1}{2} \Omega(\omega) q$ in Equation (4.7) uses the 4×4 skew-symmetric matrix:

$$\Omega(\omega) = \begin{bmatrix} -[\omega]_{\times} & \omega \\ \omega^T & 0 \end{bmatrix} \in \mathbb{R}^{4 \times 4} \quad (\text{A.3})$$

where $[\omega]_{\times}$ denotes the 3×3 skew-symmetric (cross-product) matrix of ω . This form follows directly from the quaternion product rule under the JPL convention.

A.3 Inter-Frame Baseline Estimation

The inter-frame camera baseline magnitude used in the hover detector and entry guards is estimated from the most recent clone pose and the current propagated IMU state. Let $p_{\{I,\text{prev}\}}^G$ and $p_{\{I,\text{curr}\}}^G$ denote the IMU positions at the previous and current frame timestamps respectively,

and let p_{CinI} be the camera position in the IMU frame from extrinsic calibration. The camera baseline in the global frame is:

$$b_{cam} = \|p_{\{I,curr\}^G} - p_{\{I,prev\}^G}\| \quad (A.4)$$

This scalar estimate is used both for the baseline-adaptive inlier filter selection in Section 4.8.3 and for the FIFO→LIFO entry baseline guard in Section 4.8.4.

APPENDIX B: CONFIGURATION REFERENCE

The following table maps the primary configuration keys to their corresponding system behaviour.

A complete list of all keys is maintained in `config/config_loader.py` and `msckf/VioManagerOptions.py`.

Table B.1: Key Configuration Parameters

Key	Default / Range	Effect
hovering_threshold	0.003 / $[0, \infty)$	Bearing residual threshold τ_{hover} for hover decision ξ_k
hover_smoothing_required	3 / $[1, \infty)$	Length of the smoothing deque; all must be 1 for $\xi = 1$
hover_entry_max_baseline	0.02 m	Maximum inter-frame baseline for LIFO entry guard
hover_entry_max_velocity	0.05 m/s	Maximum velocity norm for LIFO entry guard
hover_exit_velocity_threshold	0.10 m/s	Velocity hard-exit threshold from LIFO
hover_max_duration_sec	10.0 s	Maximum time in LIFO before unconditional FIFO return
hover_baseline_threshold	0.01 m	Baseline threshold separating epipolar vs. bearing filter policies
lifo_backward_min_chain_length	3	Minimum chain length for a feature to contribute LIFO updates
lifo_quality_init	0.5	Initial quality score q_i on chain creation
lifo_quality_inc_match	0.1	Quality increment Δq_{match} on successful match
lifo_quality_dec_miss	0.2	Quality decrement Δq_{miss} on missed association
lifo_quality_min_keep	0.1	Quality below which chain is pruned
lifo_quality_min_output	0.3	Quality threshold for passing feature to MSCKF update
lifo_orb_descriptor_fallback	false	Enable ORB descriptor as conditional fallback in Hungarian matching
try_zupt	true	Enable ZUPT module
zupt_chi2_multiplier	1.0	Multiplier on ZUPT chi-square acceptance threshold
zupt_max_velocity	0.1 m/s	Maximum velocity for ZUPT acceptance
zupt_max_disparity	0.5 px	Maximum visual disparity for ZUPT acceptance

integration	RK4	IMU integration method: DISCRETE / RK4 / ANALYTICAL
max_clones	15	Maximum clone window size N_max
max_slam	25	Maximum SLAM landmarks in state
use_fej	true	Enable First-Estimates Jacobians for consistency